

Structured Programming Concepts ***(26CSE0105)***



Problem Solving & Logic Development

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

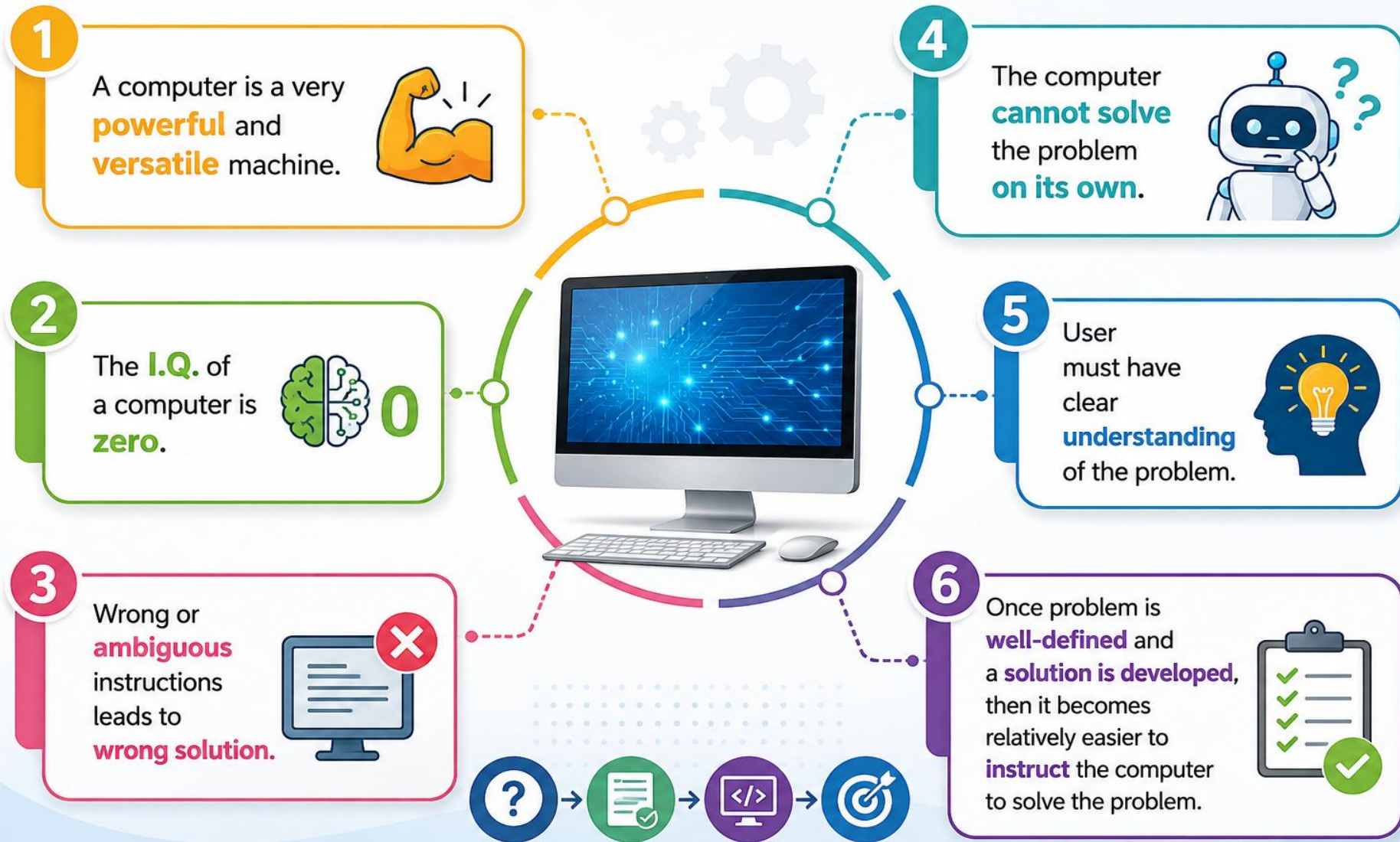
Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Problem definition***
- ***Problem analysis***
- ***General problem-solving strategies***
- ***Algorithm design and characteristics***
- ***Algorithm development techniques***

Introduction



Problem Definition



- A clear **problem definition** is the **foundation** of **successful problem solving**.



Problem = Difference between the present state and the desired state

Problem Definition: Example

- *Vague problems lead to vague solutions.*

 VAGUE STATEMENT	 WELL-DEFINED STATEMENT
"Students are performing poorly." ✗ What subject? ✗ Which students? ✗ How poor? ✗ Why?	"Average marks in Computer Networks fell from 72% to 54% in Semester III. The objective is to identify the causes and improve performance." ✓ Specific ✓ Measurable ✓ Focused

Problem Solving Steps

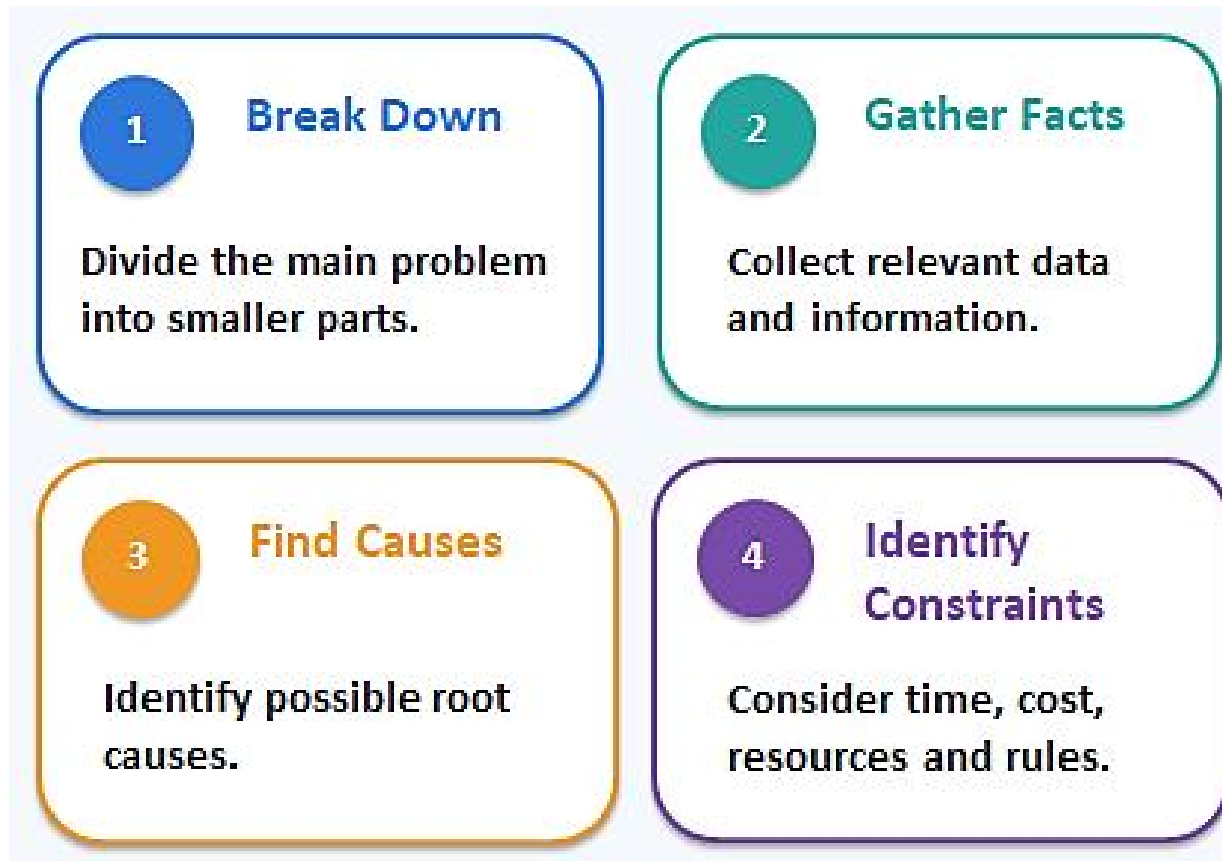


- Once the **problem is defined**, the **steps/solution required** to solve it.
- Before writing any code, the **problem must be stated precisely**: **what is given** and **what is required**.



Problem Analysis

- Once the **problem is defined**, **analyze it** before jumping to a solution.
- **Analysis breaks defined problem** into its **working fragments**.



Problem Analysis: Example

- **Problem:** *Find the greatest between three integer numbers.*

Inputs	What data must the solution accept?	<i>Three integers: a, b, c</i>
Outputs	What result must it produce?	<i>The single largest value</i>
Processing	What operations transform input to output?	<i>Compare pairs of values</i>
Constraints	What limits or rules apply?	<i>Values may be equal or negative</i>

General Problem Solving Strategies

- *Different problems may require different approaches.*

1

Trial & Error

- Try possible solutions and learn from failures.

2

Divide & Conquer

- Break a large problem into smaller, independent sub-problems.
- Solve each, then combine.

3

Top-Down Approach

- Start from the overall goal and refine it into progressively detailed steps.

4

Working Backward

- Start from the desired goal and move backward.

5

Pattern Recognition

- Recognize similarity to a previously solved problem and reuse its method.

6

Greedy Approach

- At each step choose the locally best option, aiming for a good overall result.

How to Choose a Strategy?

Large and complex problem



Divide & Conquer

Known goal but unclear path



Working Backward

Similar problem solved before



Pattern Recognition

Need a quick practical solution



Heuristics

Complete Problem-Solving Process

1

1. Define

What exactly is the problem?

2

2. Analyze

Why does it exist?

3

3. Generate

What solutions are possible?

4

4. Select

Which solution is best?

5

5. Implement

Put the solution into action.

6

6. Evaluate

Did it solve the problem?

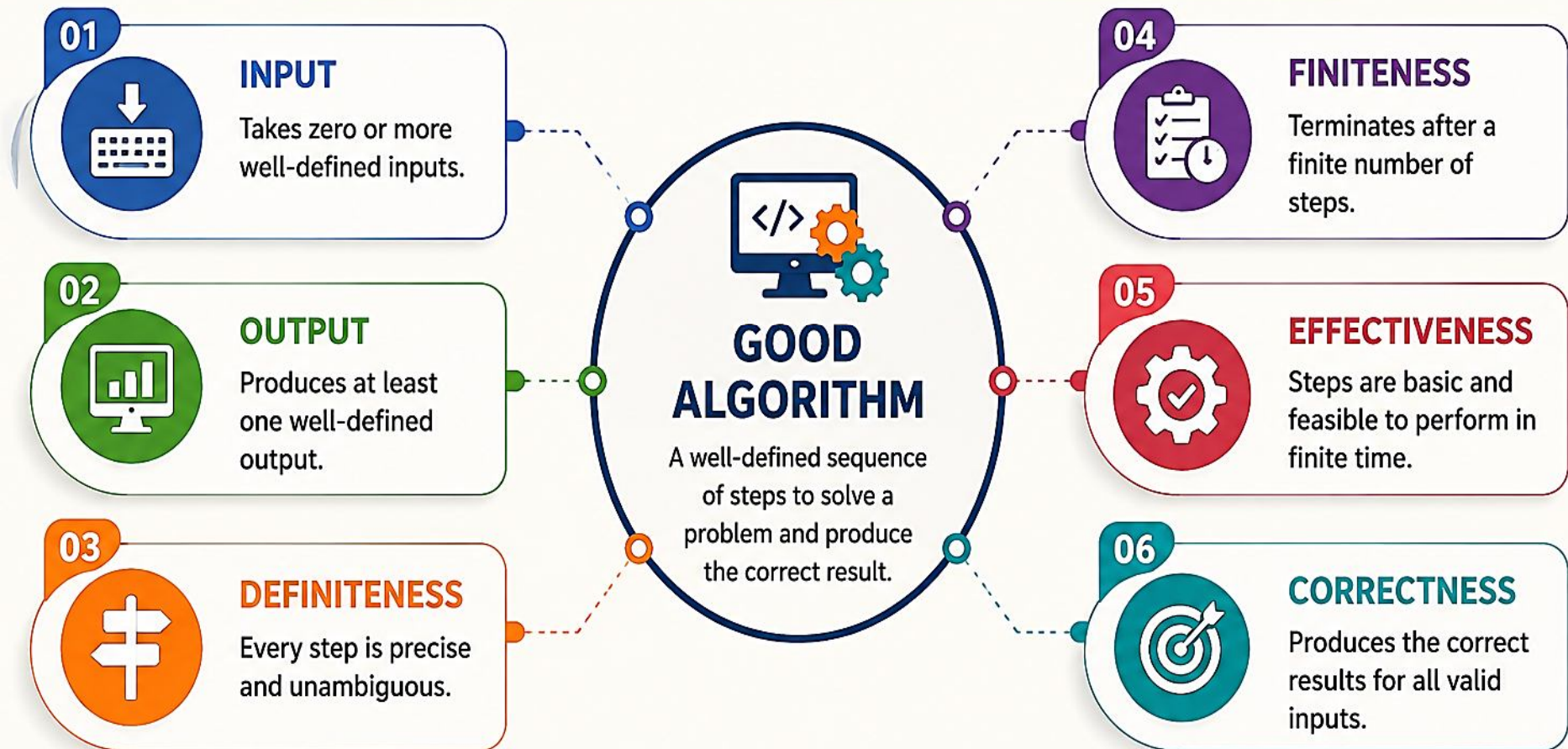
Algorithm: Definition

- An **algorithm** is a **finite**, **step-by-step**, and **well-defined** set of instructions used to **solve a problem** or **perform a task**.
- Algorithm has a **crucial role in various fields**.
- Some of the key areas are:
 - ❖ **Computer Science**
 - ❖ **Mathematics**
 - ❖ **Operations Research**
 - ❖ **AI & Data Science**

Algorithm: Characteristics



A good algorithm must satisfy the following essential properties.



Algorithm Development Methodologies



Algorithm Development Methodologies are systematic approaches and principles used to design, structure and implement algorithms in an efficient and organized manner.

1 STEPWISE REFINEMENT



Key Idea

Start with a broad outline of the solution and gradually refine each step into finer detail until it is ready to code.

How it Works

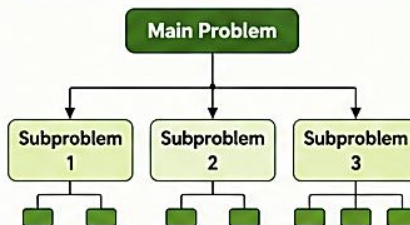
- 1 Develop a general solution outline.
- 2 Add more detail to each step.
- 3 Refine and improve the solution.
- 4 Repeat until the algorithm is precise and implementable.

Example

Finding the largest number in an array.



2 TOP-DOWN DEVELOPMENT



Key Idea

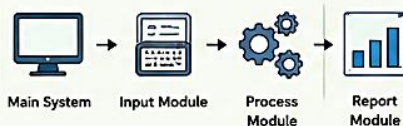
Design the main structure first, then develop each subordinate module or function separately.

How it Works

- 1 Identify the main problem.
- 2 Break it into smaller subproblems.
- 3 Solve each subproblem independently.
- 4 Integrate all modules to get the complete solution.

Example

Student Result Management System



3 MODULAR DESIGN



Key Idea

Split the solution into independent, reusable modules that each handle one responsibility.

How it Works

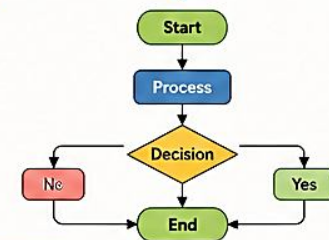
- 1 Identify distinct tasks or functions.
- 2 Design each module separately.
- 3 Ensure modules are independent and reusable.
- 4 Combine modules to form the complete algorithm.

Example

Library Management System



4 STRUCTURED PROGRAMMING



Key Idea

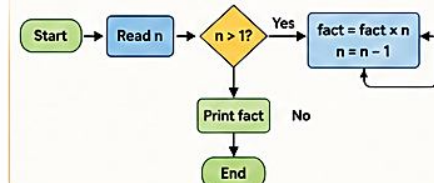
Build the algorithm using only three basic control structures: Sequence, Selection (Decision) and Iteration.

How it Works

- 1 Use sequence for step-by-step actions.
- 2 Use selection for decision making.
- 3 Use iteration for repeated actions.
- 4 Combine these structures to solve complex problems.

Example

Factorial of a Number (n!)



Algorithm Development Techniques



Different techniques and strategies used to design efficient, effective and optimized algorithms to solve a problem.

1 BRUTE FORCE



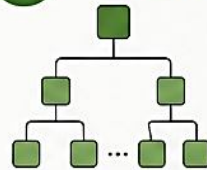
Solve the problem by trying all possible solutions and selecting the correct one.

How it works:

- Generate all possible solutions.
- Check each solution for correctness.
- Choose the valid/best solution.

Example: Linear Search, Simple Sorting

2 DIVIDE AND CONQUER



Break the problem into smaller subproblems, solve them recursively and combine their solutions.

How it works:

- Divide the problem into smaller parts.
- Solve each part (recursively).
- Combine results to get the final solution.

Example: Merge Sort, Quick Sort, Binary Search

3 GREEDY METHOD



Make the locally optimal choice at each step with the hope of finding the global optimum.

How it works:

- Make the best choice available at the moment.
- Move to the next step.
- Repeat until the solution is complete.

Example: Dijkstra's Algorithm, Kruskal's MST

4 DYNAMIC PROGRAMMING

i\j	1	2	3	4
1				
2				
3				
4				

Solve complex problems by breaking them into overlapping subproblems and storing results to avoid recomputation.

How it works:

- Break the problem into overlapping subproblems.
- Solve each subproblem once and store the result.
- Use stored results to build the final solution.

Example: Fibonacci Series, 0/1 Knapsack, Matrix Chain Multiplication

5 BACKTRACKING



Build solutions incrementally and abandon paths that cannot lead to a valid solution.

How it works:

- Build solution step by step.
- If a step leads to a dead end, backtrack.
- Try alternative choices.

Example: N-Queens Problem, Sudoku Solver, Graph Coloring

6 HEURISTIC METHOD



Use problem-specific rules or experience to find good enough solutions within reasonable time.

How it works:

- Use heuristics or rules of thumb.
- Find good (not always optimal) solution.
- Faster but may not guarantee the best result.

Example: A* Search, Genetic Algorithm, Simulated Annealing

Algorithm Development Techniques

Comparison of Development Techniques

Brute Force

Try possibilities

Simple

Divide & Conquer

Split and combine

Scalable

Greedy

Choose local best

Fast

Dynamic Programming

Store sub-results

Avoid repeated work

Backtracking

Try and undo

Explores choices

Heuristic

Use practical guidance

Good solution quickly

Algorithm to add 3 numbers and print their sum:

- **START**
- **Declare** 3 integer variables **num1**, **num2**, and **num3**.
- **Take** 3 numbers, as inputs in variables **num1**, **num2**, and **num3** respectively.
- **Declare** an integer variable **sum** to store the resultant sum of the 3 numbers.
- **Add** the 3 numbers and store the result in the variable **sum**.
- **Print** the value of the variable **sum**.
- **END**

Algorithm to find and print largest of 3 numbers:

- **START**
- **Declare** 4 integer variables **a**, **b**, **c** and **largest**.
- **Take** 3 numbers, as inputs in variables **a**, **b**, **c** respectively.
- **Compare** 3 numbers. If **a > b** and **a > c**, set **largest = a**.
- Else if **b > a** and **b > c**, set **largest = b**.
- Else set **largest = c**.
- **Print** the value of the variable **largest**.
- **END**



Thank You!

